

CHIRANJEEVI

+91 7061216213 ◇ Bhopal, Madhaya Pradesh India

◇ Email ◇ linkedin ◇ github

SUMMARY

Computer Science student with specialisation (AI&DS) with skills in Web Development, Data Structures & Algorithms, HTML, CSS, JavaScript, React, C++, and databases(Mongoose), openAI, ChatModels, Langchain, pydantic(BaseModel, EmailStr, Field), Parsers, Chains TypedDict(Annotated, Optional, Literal) passionate about building efficient, user-friendly applications and solving real-world problems through technology .

EDUCATION

B.Tech – Computer Science , Jai Narain College Of Technology, Bhopal	2023-2027
6 Year Gap (Neet Preparation, BCA - 1year)	2017-2023
12 th (Intermediate)BSEB, Mirza Ghalib college Gaya, Bihar	2017
10 th (Matriculation) BSEB, Utkramit Uchha vidyalaya sadipur Gaya, Bihar	2015

SKILLS

Language & Frameworks	javaScript, C++, React.js, Next.js, Node.js, Express.js, DSA with C++ Frontend, Langchain, Pydantic, TypedDict, Python , HTML5, CSS3, Tailwind CSS, React Hooks, Context API, Responsive Design Backend & Mongoose Atlas , ChatModels, Embeddings.
Cloud & DevOps	Netlify, Git, GitHub
Testing	Postman
Tools	VS Code, Vite, MS-Excel, MS-Word, MS-PowerPoint,
LLM Model & APIs	Google Generative AI, OpenAI, Anthropic, HuggingFace

PROJECTS

College - TimeTable Generator *React, Vite, tailwindCss, javaScript,*

Designed and developed an intelligent scheduling platform that automatically generates conflict-free semester timetables using a Backtracking-based constraint satisfaction approach. Engineered a responsive user interface with React and Tailwind CSS, enabling efficient timetable creation, faculty allocation, and schedule management while significantly reducing manual scheduling complexity.

Key points

- Developed a Smart Timetable Generator that uses a Backtracking Algorithm to automatically create conflict-free semester schedules and eliminate teacher scheduling clashes.
- Built a responsive and user-friendly web application using React, Vite, and Tailwind CSS, ensuring compatibility across multiple devices and browsers.
- Implemented PDF, Excel, and print export functionality, enabling easy timetable sharing and management.

Backend-Community-VideoStreaming

Mongoose-atlas, Node.js, javascript, Cloudinary, EC2, Redux

Community-based video Watching platform with built using Node.js, Express middleware, and MongoDB schema design with Mongoose. Used Redux and Context API for client-side state management. Secured JWT-based authentication with refresh token rotation and server-side pagination, improving API response time by 35%.

Key Points

- Tested and validated REST APIs using Postman. Developed a full-stack Smart Timetable
- Generator using the MERN stack, MongoDB-Atlas, Mongoose, Cloudinary, and Backtracking Algorithm to create conflict-free semester schedules with PDF, Excel, and print export features.

An interactive Streamlit application that generates structured, customizable summaries of research papers based on flexible Explanation Styles and Explanation Lengths. Built with LangChain (ChatOpenAI) and enforced using Pydantic schema validation.

Features

- Custom Explanation Styles: Tailor summaries for different audiences (e.g., *ELI5 / Beginner*, *Academic / Technical*, *Executive Summary*, *Analogy-based*).
- Dynamic Explanation Length: Select between short overviews, detailed bulleted breakdowns, or comprehensive deep dives.
- Guaranteed Structured Output: Leverages Pydantic BaseModel & Field annotations with typed annotations/literals to enforce consistent schema output without hallucinations.
- Powered by OpenAI: Uses ChatOpenAI for fast, contextual, and accurate model reasoning.

Tech Stack & Concepts

- Frontend/UI: Streamlit
- LLM Orchestration: LangChain (ChatOpenAI)
- Data Validation & Schemas: Pydantic (BaseModel, Field), Python TypedDict, Literal, and Type Annotations
- Deployment: Streamlit Community Cloud

How It Works

1. Input your research paper text or query.
2. Select your desired **Explanation Style** (e.g., Technical vs. ELI5) and **Length** (e.g., Short, Medium, Detailed).
3. The application constructs a typed, structured prompt, sending it to ChatOpenAI via structured outputs.
4. The output is parsed into validated Pydantic models to render a clean, human-readable summary on Streamlit.

AI User Review Analyzer & Structured Data Extractor

chatOpenAI, streamlit, dotenv_load, pydantic, TypedDict,

An interactive Streamlit application that transforms unstructured customer reviews and feedback into structured, validated analytical data. Built with LangChain (ChatOpenAI) and enforced using Pydantic data schemas for deterministic output extraction.

Features

- Structured Data Extraction: Automatically extracts key themes, concise summaries, sentiment flags, pros, cons, and reviewer metadata from raw unstructured text.
- Strict Type Enforcement: Uses Literal["pos", "neg"] to mandate precise binary sentiment classification without vague responses.
- Conditional Metadata Parsing: Employs Optional type hints to seamlessly handle optional fields like missing reviewer names or non-existent pros/cons
- Guaranteed Schema Output: Leverages Pydantic BaseModel and Field annotations with .with_structured_output() to eliminate raw string parsing errors and LLM hallucinations

Tech Stack & Concepts

- Frontend/UI: Streamlit
- LLM Orchestration: LangChain (ChatOpenAI)
- Data Validation & Schemas: Pydantic (BaseModel, Field), Python typing (Literal, Optional)
- Environment & Config: python-dotenv

How It Works (Not Deployed Yet)

1. Input raw customer review text into the interactive Streamlit text area.
2. The input is passed through ChatOpenAI wrapped in a Pydantic Review schema via `.with_structured_output()`.
3. The LLM validates the text against the model's type constraints (Literal, list[str], Optional[str]).
4. The validated result is returned as a native Python object and rendered cleanly onto the Streamlit dashboard with conditional UI sections.